LAPORAN PRAKTIKUM
POSTTEST 6
ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Ananda Rizki Apriatama (2609106007)

Kelas A1 ‘26

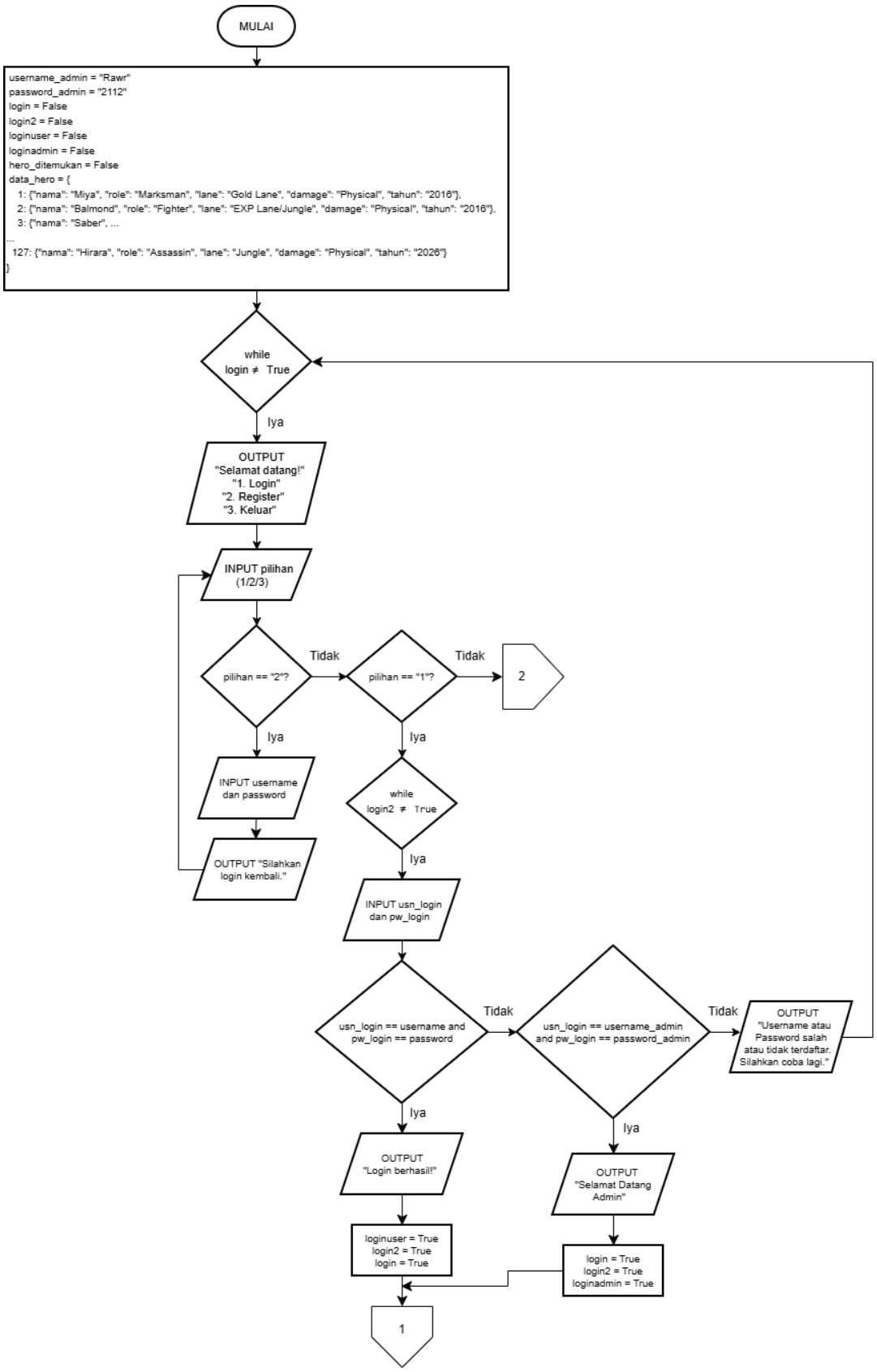
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

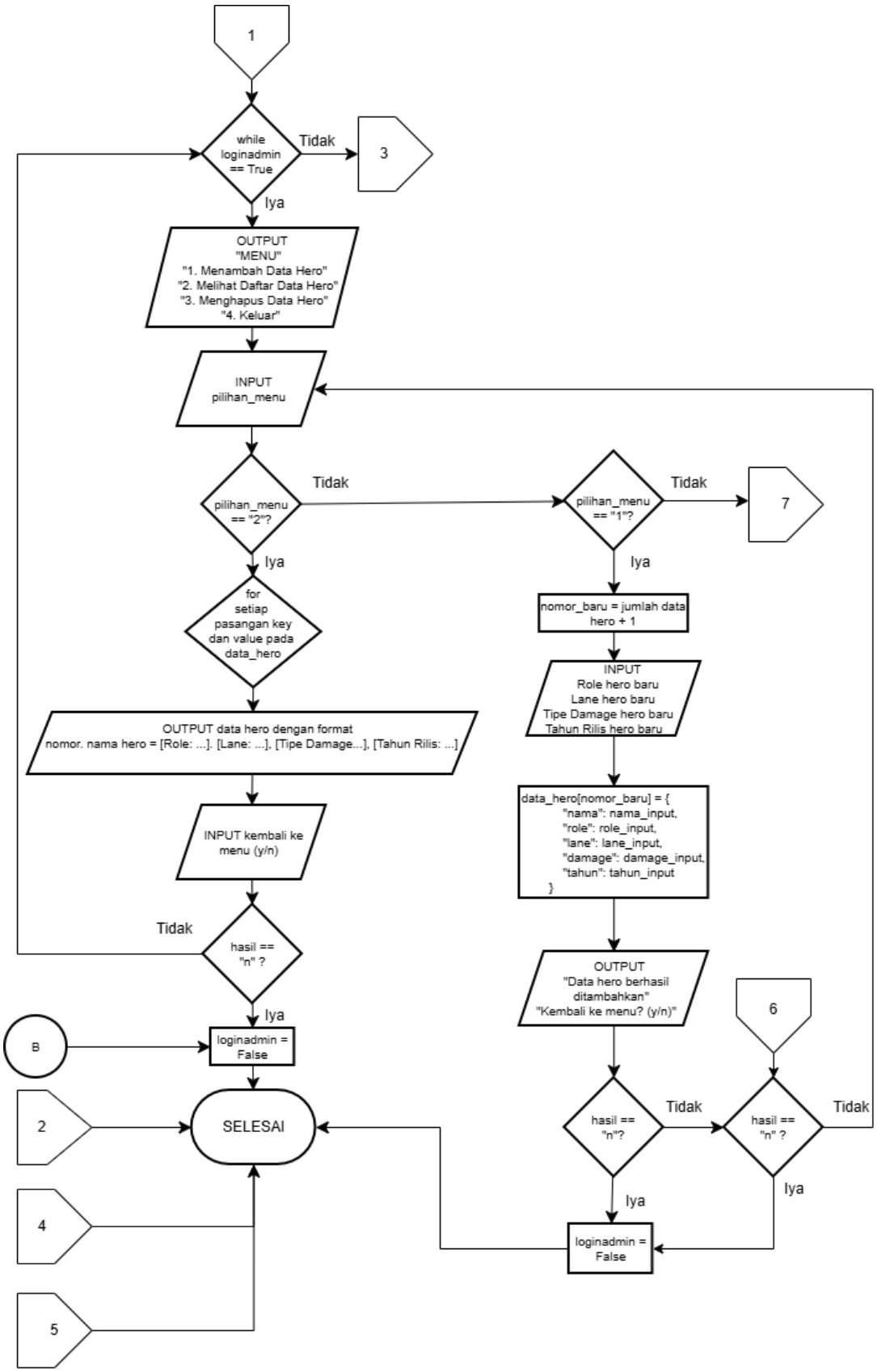
SAMARINDA

2026

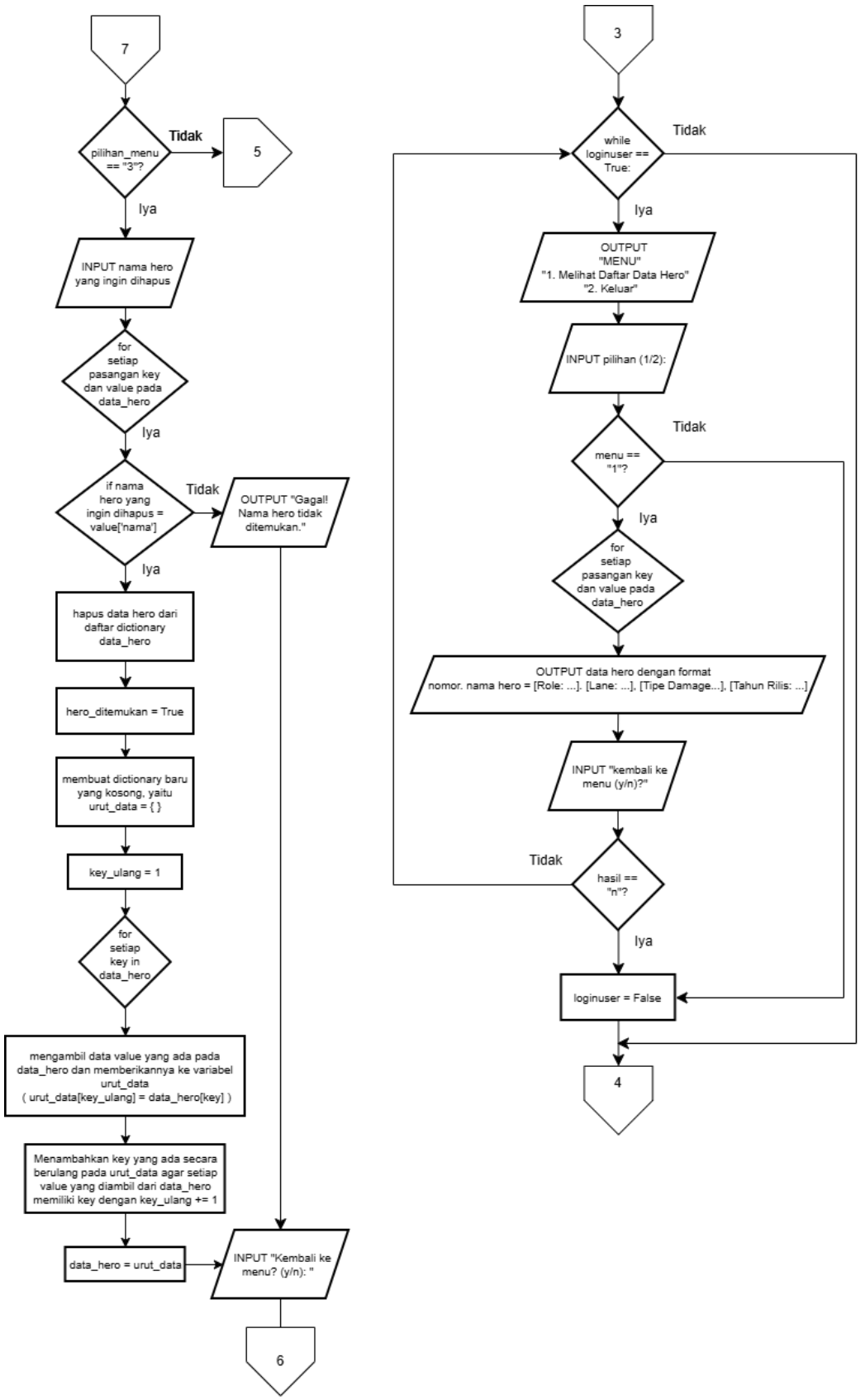
1. Flowchart



Gambar 1.1 *Flowchart Page 1 Program Manajemen Data Hero Mobile Legends*



Gambar 1.2 *Flowchart Page 2 Program Manajemen Data Hero Mobile Legends*



Gambar 1.3 *Page 3 Flowchart Program Manajemen Data Hero Mobile Legends*

2. Deskripsi Singkat Program

Program ini adalah manajemen data hero Mobile Legends berbasis yang dibuat dengan Python. Program memiliki sistem login dan register dengan dua jenis akses, yaitu pengguna biasa dan admin. Pengguna biasa hanya dapat melihat daftar data hero, sedangkan admin dapat melihat, menambah, dan menghapus data hero. Data hero disimpan dalam dictionary yang berisi nama, role, lane, tipe damage, dan tahun rilis. Program berjalan terus-menerus dengan perulangan while sampai pengguna memilih keluar.

3. Source Code

A. Inisialisasi Variabel

Baris pertama meng-import modul os agar layar terminal bisa dibersihkan dengan os.system. Setelah itu, menginisialisasi variabel yang dibutuhkan untuk akun admin bawaan, yaitu username_admin dan password_admin. Serta, variabel-variabel lainnya yang akan digunakan pada program-program/kode-kode berikutnya.

Source Code:

```
import os
username_admin = "Rawr"
password_admin = "2112"
login = False
login2 = False
loginuser = False
loginadmin = False
hero_ditemukan = False
```

B. Data Hero

Variabel data_hero berupa nested dictionary. Setiap key-nya adalah nomor urut hero (1 sampai 127), dan value-nya adalah dictionary lain yang menyimpan nama, role, lane, tipe damage, dan tahun rilis hero.

Source Code:

```
data_hero = {
```

```
    1: {"nama": "Miya", "role": "Marksman", "lane": "Gold Lane",
"damage": "Physical", "tahun": "2016"},
    2: {"nama": "Balmond", "role": "Fighter", "lane": "EXP
Lane/Jungle", "damage": "Physical", "tahun": "2016"},
    3: {"nama": "Saber", "role": "Assassin", "lane": "Jungle",
"damage": "Physical", "tahun": "2016"},
    4: {"nama": "Alice", "role": "Mage/Tank", "lane": "EXP Lane",
"damage": "Magic", "tahun": "2016"},
    5: {"nama": "Nana", "role": "Mage/Support", "lane": "Mid Lane",
"damage": "Magic", "tahun": "2016"},
    6: {"nama": "Tigreal", "role": "Tank", "lane": "Roam", "damage":
"Physical", "tahun": "2016"},
    7: {"nama": "Alucard", "role": "Fighter/Assassin", "lane":
"Jungle", "damage": "Physical", "tahun": "2016"},
    8: {"nama": "Karina", "role": "Assassin", "lane": "Jungle",
"damage": "Magic", "tahun": "2016"},
    9: {"nama": "Akai", "role": "Tank", "lane": "Roam/Jungle",
"damage": "Physical", "tahun": "2016"},
    10: {"nama": "Franco",...
...
  127: {"nama": "Hirara", "role": "Assassin", "lane": "Jungle",
"damage": "Physical", "tahun": "2026"}
}
```

C. Laman Awal

Bagian ini menampilkan menu awal berisi pilihan Login, Register, dan Keluar, yang diulang dengan while selama login belum berhasil. Pada pilihan Register, pengguna membuat username dan password baru. Pada pilihan Login, program membandingkan input dengan data akun pengguna atau akun admin. Jika cocok dengan akun pengguna, variabel loginuser menjadi True. Jika cocok dengan akun admin, variabel loginadmin menjadi True. Jika tidak cocok dengan keduanya, program menampilkan pesan bahwa username/password salah atau akun tidak terdaftar lalu kembali ke laman awal. Pilihan selain 1 dan 2 menghentikan program.

Source Code:

```
os.system('cls' if os.name == 'nt' else 'clear')
```

```
while login != True:
    print('='*22, "Selamat datang!", "="*22)
    print("1. Login")
    print("2. Register")
    print("3. Keluar")
    pilihan = input("Masukkan Pilihan (1/2/3): ")
    if pilihan == "2":
        os.system('cls' if os.name == 'nt' else 'clear')
        print('='*22, 'LAMAN REGISTER', '='*22)
        username = input('Buat Username: ')
        password = input('Buat Password: ')
        print("Silahkan login kembali.")
    elif pilihan == "1":
        while login2 != True:
            os.system('cls' if os.name == 'nt' else 'clear')
            print('='*22, 'LOGIN', '='*22)
            usn_login = input('Masukkan Username: ')
            pw_login = input('Masukkan Password: ')
            if usn_login == username and pw_login == password:
                os.system('cls' if os.name == 'nt' else 'clear')
                print('='*22, 'Login berhasil.', '='*22)
                loginuser = True
                login2 = True
                login = True
                break
            elif usn_login == username_admin and pw_login ==
password_admin:
                os.system('cls' if os.name == 'nt' else 'clear')
                print('='*22, 'Selamat Datang Admin.', '='*22)
                login = True
                login2 = True
                loginadmin = True
                break
            elif usn_login != username or pw_login != password:
                os.system('cls' if os.name == 'nt' else 'clear')
                print('Username atau Password salah atau tidak
terdaftar! Silahkan coba lagi.')
```

```
                break
    else:
        break
```

D. Menu Pengguna (Non-Admin)

Menu ini tampil ketika loginuser bernilai True dan hanya punya dua pilihan: melihat daftar data hero atau keluar. Pada pilihan melihat daftar, program menampilkan seluruh isi data_hero dengan perulangan for dan print f-string. Setelah itu pengguna ditanya apakah ingin kembali ke menu (y/n). Jika memilih "n" atau memilih keluar, program akan berhenti.

Source Code:

```
while loginuser == True:
    os.system('cls' if os.name == 'nt' else 'clear')
    print('='*44)
    print('='*19, 'MENU', '='*19)
    print('='*44)
    print('1. Melihat Daftar Data Hero')
    print('2. Keluar')
    menu = input('Masukkan Pilihan (1/2): ')
    if menu == "1":
        os.system('cls' if os.name == 'nt' else 'clear')
        for key, value in data_hero.items():
            print(f"{key}. {value['nama']} = [Role: {value['role']}],
[Lane: {value['lane']}], [Tipe Damage: {value['damage']}], [Tahun
Rilis: {value['tahun']}]")
        print('='*44)
        hasil = input("Kembali ke menu? (y/n): ")
        if hasil == "n":
            os.system('cls' if os.name == 'nt' else 'clear')
            login = False
            break
    else:
        os.system('cls' if os.name == 'nt' else 'clear')
        loginuser = False
        break
```

E. Menu Admin: Melihat dan Menambah Data Hero

Menu admin tampil ketika loginadmin bernilai True dan memiliki empat pilihan. Pilihan 2 menampilkan seluruh daftar hero seperti pada menu pengguna. Pilihan 1 menambah hero baru: nomor urutnya dibuat dari len(data_hero) + 1, lalu admin mengisi nama, role, lane, tipe damage, dan tahun rilis, yang disimpan sebagai dictionary baru di data_hero. Setelah selesai, admin ditanya apakah ingin kembali ke menu.

Source Code:

```
while loginadmin == True:
    os.system('cls' if os.name == 'nt' else 'clear')
    print('='*44)
    print('='*19, 'MENU', '='*19)
    print('='*44)
    print('1. Menambah Data Hero')
    print('2. Melihat Daftar Data Hero')
    print('3. Menghapus Data Hero')
    print('4. Keluar')
    menu = input('Masukkan Pilihan (1/2/3/4): ')
    if menu == "2":
        os.system('cls' if os.name == 'nt' else 'clear')
        for key, value in data_hero.items():
            print(f"{key}. {value['nama']} = [Role: {value['role']}],
[Lane: {value['lane']}], [Tipe Damage: {value['damage']}], [Tahun
Rilis: {value['tahun']}]")
        print('='*44)
        hasil = input("Kembali ke menu? (y/n): ")
        if hasil == "n":
            os.system('cls' if os.name == 'nt' else 'clear')
            login = False
            break
    elif menu == "1":
        os.system('cls' if os.name == 'nt' else 'clear')
        print('='*22, 'TAMBAH DATA HERO', '='*22)
        nomor_baru = len(data_hero) + 1
        nama_input = input("Masukkan Nama Hero: ")
        role_input = input("Masukkan Role Hero: ")
```

```
        lane_input = input("Masukkan Lane Hero: ")
        damage_input = input("Masukkan Tipe Damage: ")
        tahun_input = input("Masukkan Tahun Rilis: ")
        data_hero[nomor_baru] = {
            "nama": nama_input,
            "role": role_input,
            "lane": lane_input,
            "damage": damage_input,
            "tahun": tahun_input
        }
        print('='*44)
        print("Data hero berhasil ditambahkan.")
        hasil = input("Kembali ke menu? (y/n): ")
        if hasil == "n":
            os.system('cls' if os.name == 'nt' else 'clear')
            login = False
            break
```

F. Menu Admin: Menghapus Data Hero dan Keluar

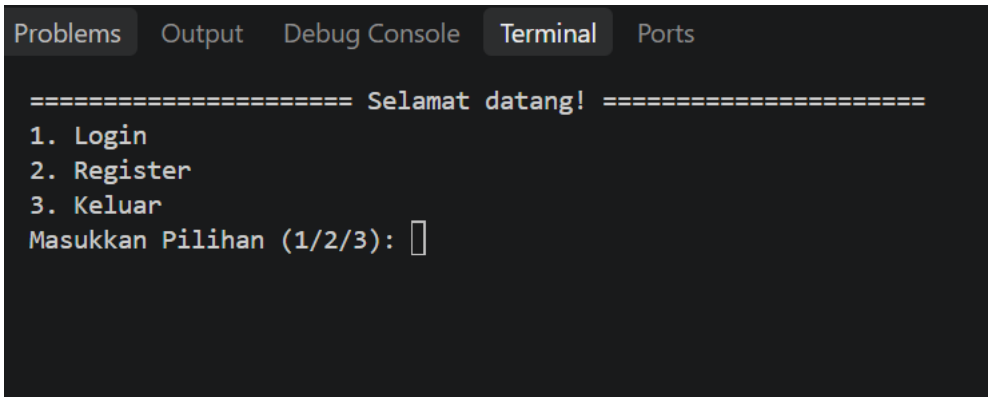
Pada pilihan 3, admin memasukkan nama hero yang ingin dihapus. Program mencari nama tersebut di data_hero dengan perulangan for. Jika ditemukan, data dihapus dengan del dan nomor urut data diurutkan ulang dari 1 lewat dictionary baru (urut_data), sehingga tidak ada nomor yang loncat. Jika tidak ditemukan, program menampilkan pesan gagal. Pilihan selain 1, 2, dan 3 (yaitu keluar) mengembalikan program ke laman awal.

Source Code:

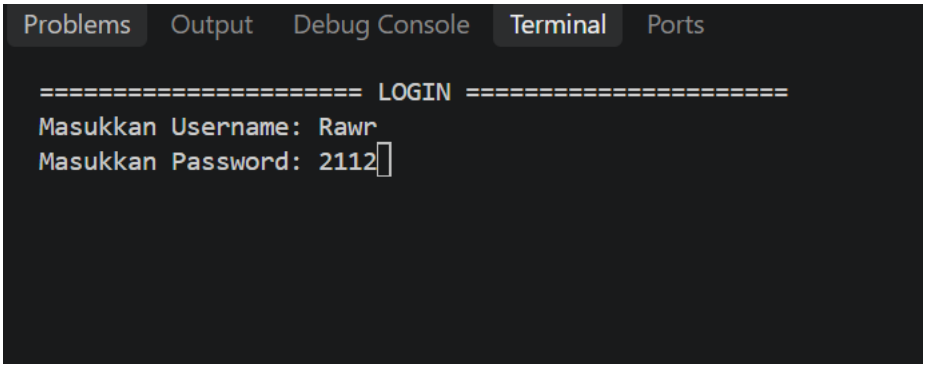
```
    elif menu == "3":
        os.system('cls' if os.name == 'nt' else 'clear')
        nama_hapus = input("Masukkan Nama Hero yang ingin dihapus
(Contoh: Miya): ")
        for key, value in data_hero.items():
            if nama_hapus == value['nama']:
                key_hapus = key
                nama_target = value['nama']
                hero_ditemukan = True
                break
```

```
        if hero_ditemukan == True:
            del data_hero[key_hapus]
            print('='*44)
            print("Sukses menghapus data:", nama_target)
            urut_data = {}
            key_ulang = 1
            for key in data_hero.keys():
                urut_data[key_ulang] = data_hero[key]
                key_ulang += 1
            data_hero = urut_data
        else:
            print('='*44)
            print("Gagal! Nama hero tidak ditemukan.")
        hasil = input("Kembali ke menu? (y/n): ")
        if hasil == "n":
            os.system('cls' if os.name == 'nt' else 'clear')
            login = False
            break
    else:
        os.system('cls' if os.name == 'nt' else 'clear')
        login = False
        break
```

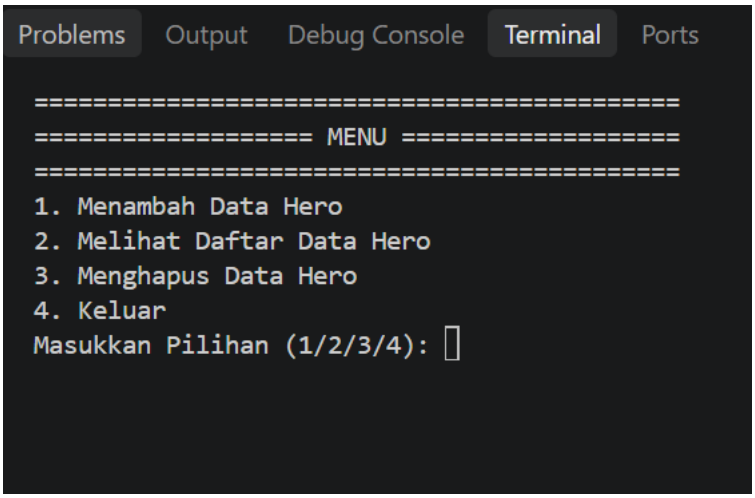
4. Hasil Output



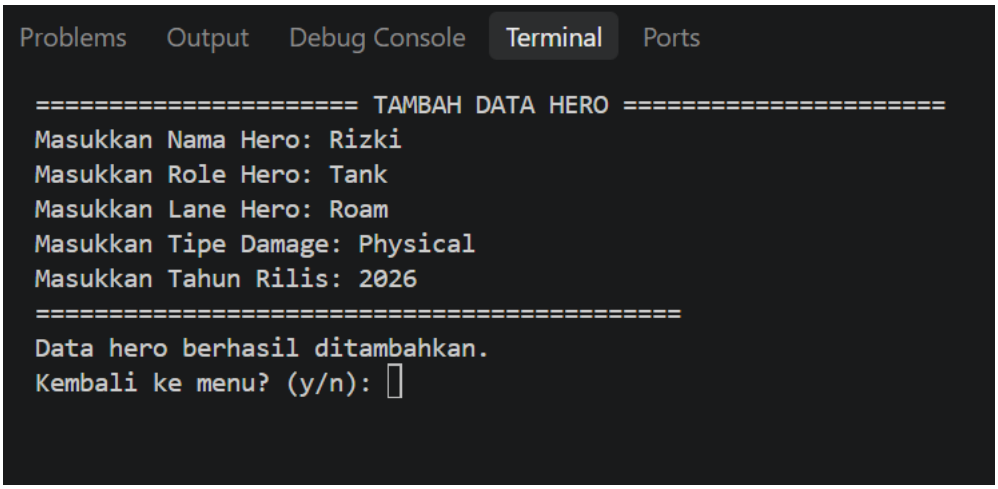
Gambar 4.1 Tampilan Laman Awal



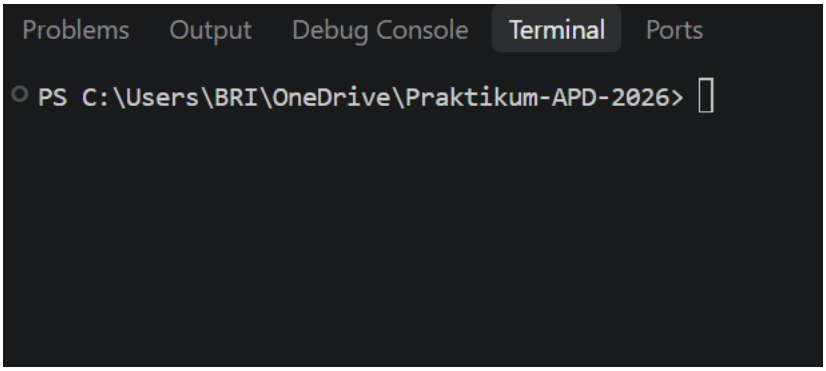
Gambar 4.2 Tampilan Laman Login



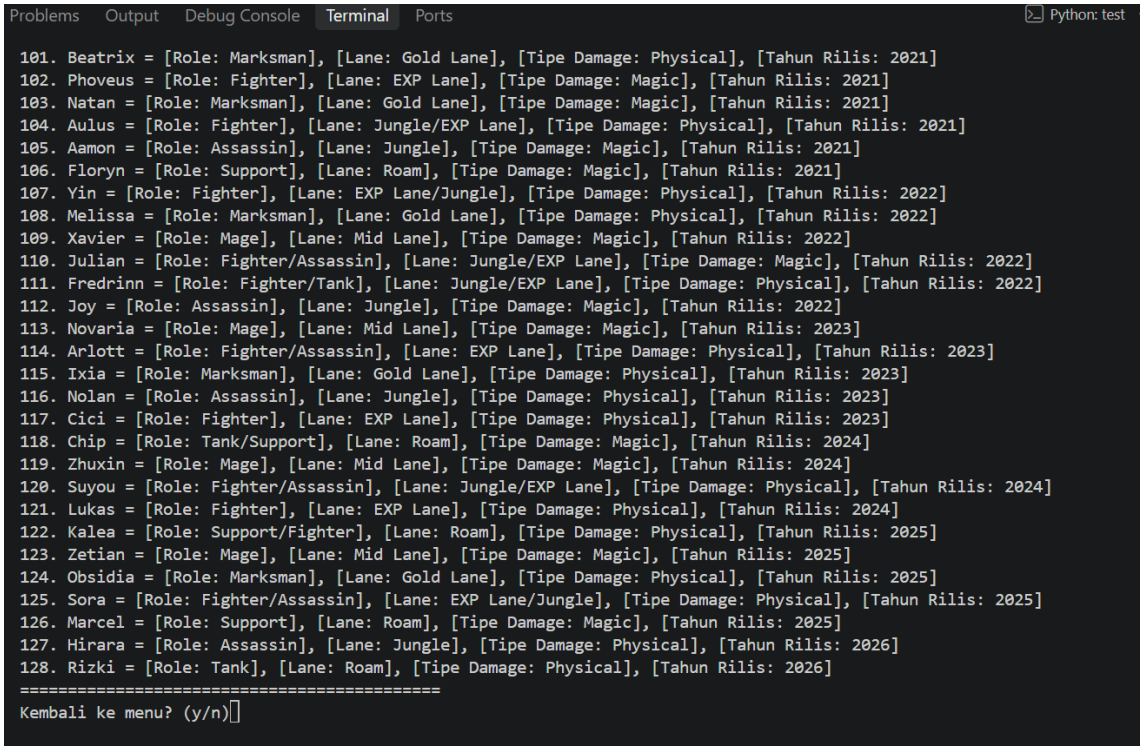
Gambar 4.3 Tampilan Menu sebagai Admin



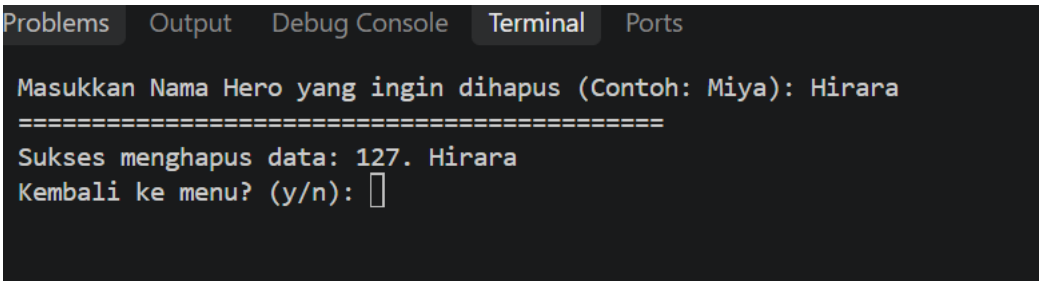
Gambar 4.4 Tampilan Menu 1



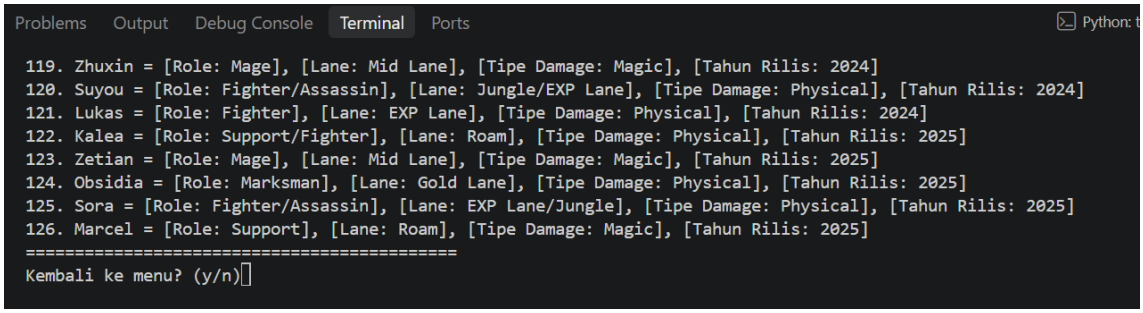
Gambar 4.5 Tampilan untuk Setiap Pilihan/Input “n” dan Pilihan Keluar Program



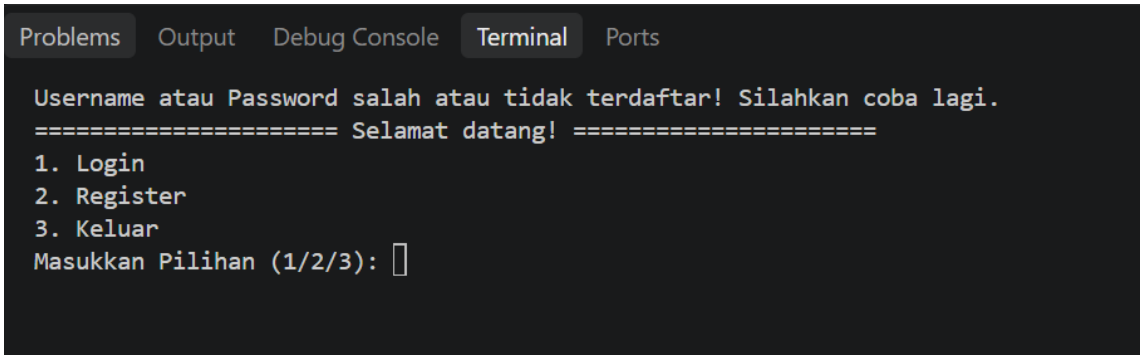
Gambar 4.6 Tampilan Menu 2, Termasuk setelah Penambahan Data Hero di Menu 1



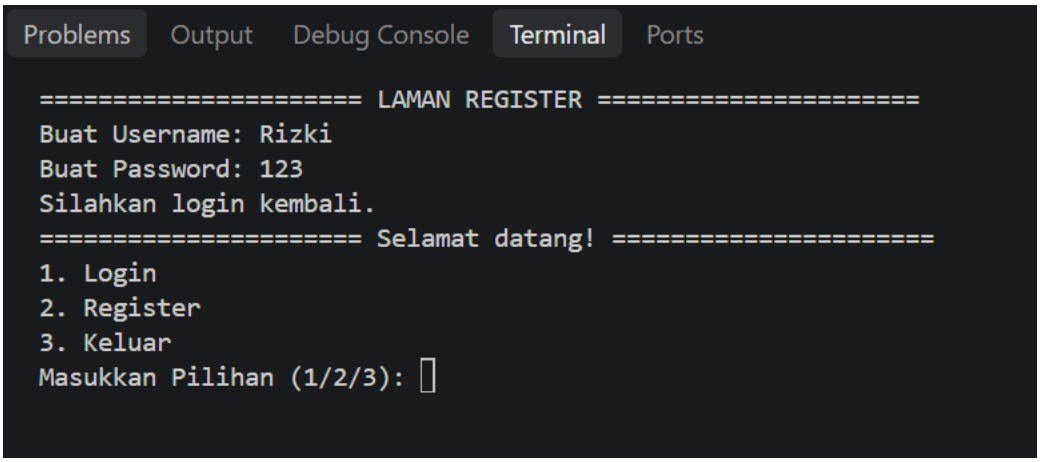
Gambar 4.7 Tampilan Menu 3



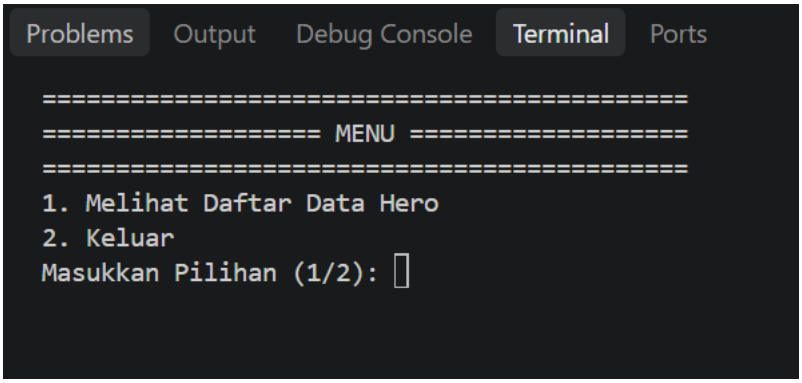
Gambar 4.8 Tampilan Menu 2 setelah Penghapus Data Hero di Menu 3



Gambar 4.9 Tampilan Laman Awal jika Login ketika Username dan Password Belum Terdaftar



Gambar 4.10 Tampilan Laman Register



Gambar 4.11 Tampilan Menu untuk Non-Admin

5. Langkah-Langkah Git

5.1 Git *Add*

```
PS C:\Users\BRI> git add.
```

Gambar 5.1 git *add* dalam terminal

Perintah git *add* digunakan untuk menambahkan *file* ke dalam *staging* area, yaitu tempat sementara sebelum *file* benar-benar disimpan lewat *commit*. Kalau mau menambahkan satu *file*, pakai *git add nama_file*. Kalau mau menambahkan semua *file* dan perubahan sekaligus di dalam folder proyek, pakai "git add .". Setelah itu bisa dicek dengan git status untuk melihat apakah *file* sudah terdaftar atau belum.

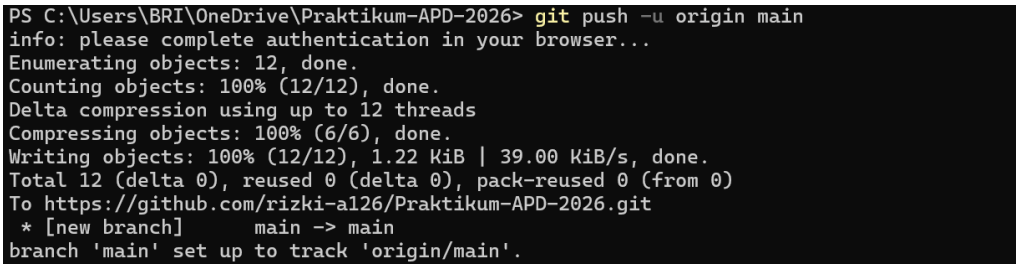
5.2 Git *Commit*

```
PS C:\Users\BRI\OneDrive\Praktikum-APD-2026> git commit -m "UP POSTTEST 6"
[main a1b2c3d4] UP POSTTEST 6
 10 files changed, 150 insertions(+), 20 deletions(-)
 create mode 100644 Kelas/test.py
 create mode 100644 Kelas/test3
 create mode 100644 posttest/posttest-apd-6/27061070008-ANANDA-RIZKI-APRIATAMA-PT-6.py
 delete mode 100644 test.py
```

Gambar 5.2 git *commit* setelah git *add* dalam terminal

Perintah git *commit* digunakan untuk menyimpan *snapshot* dari proyek, jadi setiap kali *commit* dilakukan, riwayat perubahan yang sudah dibuat ikut tersimpan. Caranya dengan mengetik *git commit -m "pesan yang ingin ditulis"*. Pesan ini berfungsi sebagai konfirmasi atau catatan tentang perubahan apa yang dilakukan pada *commit* tersebut.

5.3 Git *Push*



Gambar 5.3 *git push* setelah *git commit* dalam terminal

Perintah git *push* digunakan untuk mengunggah perubahan yang sudah disimpan secara lokal ke Github, dengan format “git push -u origin main”. Sebelumnya, *repository* lokal perlu dihubungkan dulu ke Github pakai *git remote add origin <link-repo-github>*, agar perubahan bisa dilakukan lewat terminal tanpa perlu *upload* manual. Setelah *push* berhasil, *file* yang tadinya cuma ada di komputer jadi bisa diakses dari mana saja lewat Github.